

[← Back to Articles](#)

🔗 Granite 4.1 LLMs: How They're Built

Enterprise**Article**

Published April 29, 2026

▲ Upvote **76****Yousaf Shah**[yousafshah](#)

Follow



ibm-granite

An in-depth technical walkthrough of data engineering, pre-training, supervised fine-tuning, and reinforcement learning behind the Granite 4.1 LLMs.

Authors: Granite Team, IBM

TL;DR — Granite 4.1 is a family of dense, decoder-only LLMs (3B, 8B, and 30B) trained on ~15T tokens using a multi-stage pre-training pipeline, including long-context extension of up to 512K tokens. The models are further refined with supervised fine-tuning on ~4.1M high-quality curated samples and reinforcement learning via on-policy GRPO with DAPO loss ([Yu et al., 2025](#)). Notably, the 8B instruct model matches or surpasses the previous Granite 4.0-H-Small (32B-A9B MoE) despite using a simpler dense architecture with fewer parameters. All Granite 4.1 models are released under the Apache 2.0 license.

Links:

- [Granite 4.1 HF Collection](#)

- [GitHub Repository](#)
 - [Granite Docs](#)
-

[Overview](#)

Building high-quality small language models goes beyond simply scaling compute—it requires rigorous data curation throughout training. For Granite 4.1, we prioritized data quality over quantity, progressively refining the data mixture across five pre-training stages. We further curated supervised fine-tuning data using an LLM-as-Judge framework and applied a multi-stage reinforcement learning pipeline to systematically strengthen performance in math, coding, instruction following, and general chat.

[Model Architecture](#)

Granite 4.1 models use a decoder-only dense transformer architecture. The core design choices include **Grouped Query Attention (GQA)**, **Rotary Position Embeddings (RoPE)**, **SwiGLU activations**, **RMSNorm**, and **shared input/output embeddings**.

Component	3B Dense	8B Dense	30B Dense
Embedding size	2560	4096	4096
Number of layers	40	40	64
Attention head size	64	128	128
Number of attention heads	40	32	32

Component	3B Dense	8B Dense	30B Dense
Number of KV heads	8	8	8
MLP hidden size	8192	12800	32768
MLP activation	SwiGLU	SwiGLU	SwiGLU
Position embedding	RoPE	RoPE	RoPE

All three model sizes share the same training pipeline and data strategy, differing only in architecture dimensions.

Pre-Training

Granite 4.1 is trained from scratch on approximately 15 trillion tokens using a five-phase training strategy. Phases 1–2 focus on foundational pre-training, phases 3–4 perform mid-training with progressively higher-quality data annealing, and phase 5 introduces long-context training, extending the context window to 512K tokens. Each phase employs a distinct data mixture and learning-rate schedule, gradually shifting from broad web-scale data to more curated, domain-specific content.



Figure 2: The five-phase pre-training pipeline. Phases 1–2 are pre-training, Phases 3–4 are mid-training (high-quality data annealing), and Phase 5 is long context training (LCE).

Phase 1: General Pre-Training (10T tokens)

The first phase establishes broad language understanding using a general mixture of training data with a power learning rate schedule and warmup.

Data composition:

- **CommonCrawl** ~59% — general web data
- **Code** ~20% — programming languages and repositories
- **Math** ~7% — mathematical reasoning data
- **Technical** ~10.5% — scientific papers, technical documentation and manuals
- **Multilingual** ~2% — non-English language data
- **Domain Specific** ~1.5% — domain-specific content

🔗 Phase 2: Math/Code Pre-Training (2T tokens)

Phase 2 sharply increases the proportion of code and mathematical data, pivoting toward stronger reasoning capabilities while still maintaining general language coverage.

Data composition:

- **Math** ~35% — a 5x increase over Phase 1
- **Code** ~30% — a 1.5x increase
- **CommonCrawl-HQ** ~12% — high-quality common crawl subset
- **Synthetic** ~9% — synthetic high-quality data
- **Technical** ~10%
- **Multilingual** ~3%
- **Domain** ~1%

🔗 Phase 3: High-Quality Data Annealing (2T tokens)

Phase 3 transitions into **mid-training** with a more balanced, high-quality mixture and an exponential decay learning rate schedule. This is where we start blending in chain-of-thought and synthetic instruction data.

Data composition:

- CommonCrawl-HQ ~16.67%
- Math ~16.67%
- Code ~16.67%
- Synthetic ~8.5%
- Technical ~12.5%
- Multilingual ~4.5%
- Long Chain-of-Thought ~12.5% — reasoning trajectories
- Language Instructions ~7.5% — instruction tuning data
- Code Instructions ~4.5% — instruction tuning data

🔗 Phase 4: High-Quality Data Annealing — Refinement (0.5T tokens)

The fourth phase continues mid-training with a linear learning rate decay to zero, focusing the model on the highest-quality data available.

Data composition:

- CommonCrawl-HQ ~40%
- Code ~20%
- Math ~20%
- Long Chain-of-Thought ~6%
- Code Instructions ~5%
- Language Instructions ~9%

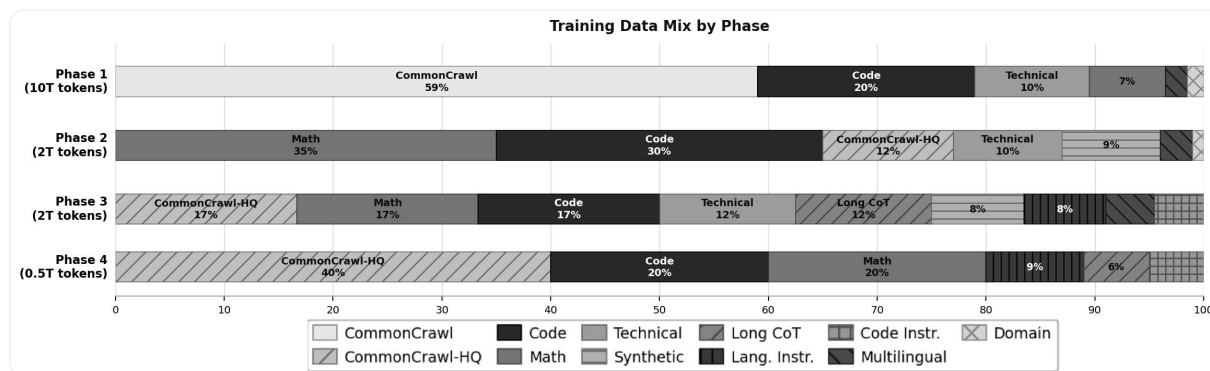


Figure 3: How the data mix evolves across the pre-training phases. Notice the progressive shift from web-heavy (Phase 1) to quality-heavy with instruction and reasoning data (Phases 3–4).

🔗 Phase 5: Long Context Training (LCE)

The fifth and final phase also part of of mid-training extends the context window from 4K to 512K through a staged long-context extension process:

1. **32K extension** — using the same data mix as Phase 4
2. **128K extension** — same data mix as Phase 4
3. **512K extension** — 80% books + 20% code repository data (8b and 30b only)

The LCE phase uses an exponential learning rate schedule starting at $1e-4$ and decaying to 0 . To ensure the model natively handles long sequences without degrading short-context performance, we do a model merge after each LCE stage. RULER benchmark of base models:

Model name	32K	64K	128K
granite-4.1-3b-base	75.0	66.6	58.0
granite-4.1-8b-base	83.6	79.1	73.0
granite-4.1-30b-base	85.2	84.6	76.7

🔗 SFT: Data Preparation & Quality Control

Supervised fine-tuning (SFT) is what turns the base model into a reliable instruction-following assistant, making data quality critically important—since even a small number of incorrect or hallucinated samples can instill undesirable behaviors. To address this, we apply a rigorous LLM-as-Judge framework alongside rule-based filtering to curate high-quality samples. Together, the pipeline automatically assess each sample against structural, semantic, and behavioral criteria, fixing issues when possible and filtering out samples that fail to meet our quality standards.

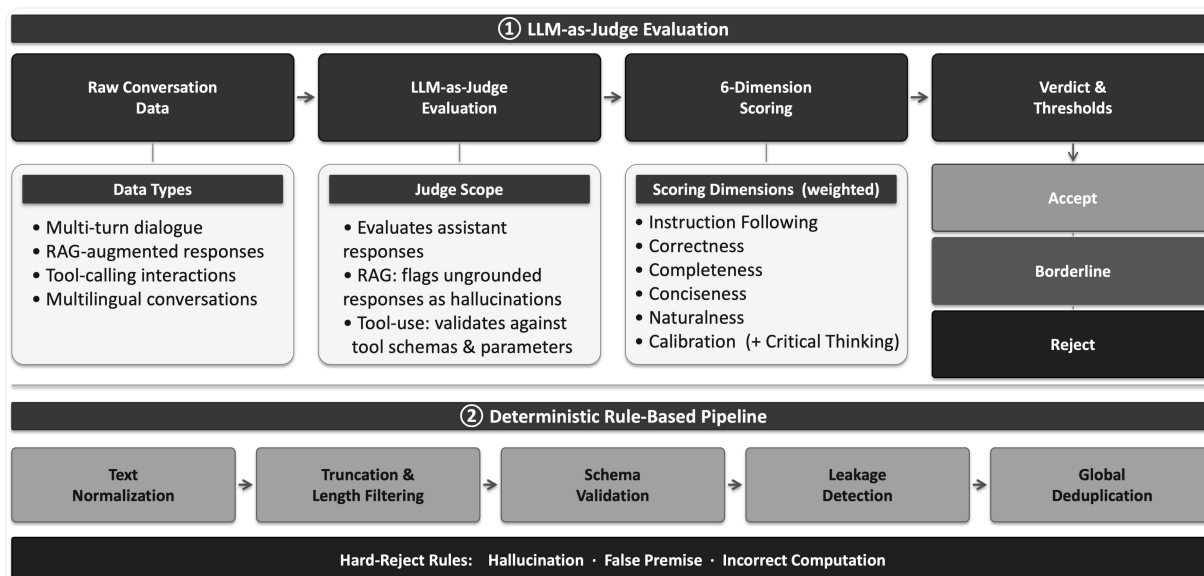


Figure 4: The SFT data quality pipeline. Raw conversation data passes through an LLM-as-Judge with a multi-dimensional rubric, producing accept/borderline/reject verdicts. Hard-reject defects (hallucination, false premise, incorrect computation) trigger automatic rejection regardless of score.

Our rigorous LLM-as-Judge framework evaluates only assistant responses, treating system prompts, user inputs, retrieved documents, and tool outputs strictly as contextual information. This ensures that the judge assesses what the model says, rather than what it was asked to do. In RAG settings, responses that are not grounded in the retrieved context are

flagged as hallucinations, while tool-use outputs are validated against the set of allowed tools and their parameter schemas.

We employ specialized judge prompts tailored to different SFT data types, including multi-turn dialogue, RAG-augmented responses, tool-calling interactions, and multilingual conversations. Each response is scored across six weighted dimensions—instruction following, correctness, completeness, conciseness, naturalness, and calibration (with optional critical-thinking checks). Samples are accepted, flagged as borderline, or rejected based on deterministic score thresholds, with hard-reject rules overriding scores for severe defects such as hallucinations, false premises, or incorrect computations.

To complement semantic evaluation, we apply a deterministic rule-based pipeline that enforces structural integrity through text normalization, truncation and length filtering, schema validation, and leakage detection. A final global deduplication step ensures dataset-wide uniqueness. All filtering and correction actions are fully auditable.

 SFT Training Details

After passing through the LLM-as-Judge, rule-based filtering, and global deduplication pipeline, we fine-tune base models on these approximately **4.1 million** high-quality samples. The following details apply to all three model variants:

Training Configuration:

Parameter	Value
Compute	16 nodes, 4x GB200 per node
Epochs	3
Learning rate	5e-6 (linear warmup 3%, linear decay over ~25K steps)

Parameter	Value
Sequence length	16,384 tokens
Total samples	~4.1M
Effective batch size	256 samples/iter (~4.2M tokens/iter)

🔗 Reinforcement Learning: Multi-Stage RL Pipeline

After SFT, we apply a multi-stage reinforcement learning pipeline to further improve the model's capabilities across specific domains. Rather than a single RL pass, we run **multiple targeted RL stages**, each optimizing for different capabilities.

🔗 Training Methodology

We use On-policy GRPO (Group Relative Policy Optimization) ([Shao et al., 2024](#)) with DAPO (Decoupled Clip and Dynamic sAmpling Policy Optimization) loss ([Yu et al., 2025](#)) which provides more stable training signals compared to standard GRPO. However, due to computationally intensive nature of dynamic sampling, we switch it off during our training runs.

🔗 RL training configuration

Parameter	Value
Algorithm	On-policy GRPO with DAPO loss

Parameter	Value
Training stack	SkyRL (NovaSky-AI, 2025)
Samples per prompt	16
Train batch size	1024
Context length	8,192

[🔗](#) **RL Pipeline**

[Figure 10](#) depicts our Reinforcement Learning pipeline for training Granite 4.1 models. Through extensive experimentation with a variety of reinforcement learning recipes, we found that this sequence of steps minimizes catastrophic forgetting while simultaneously maximizing performance across multiple domains.

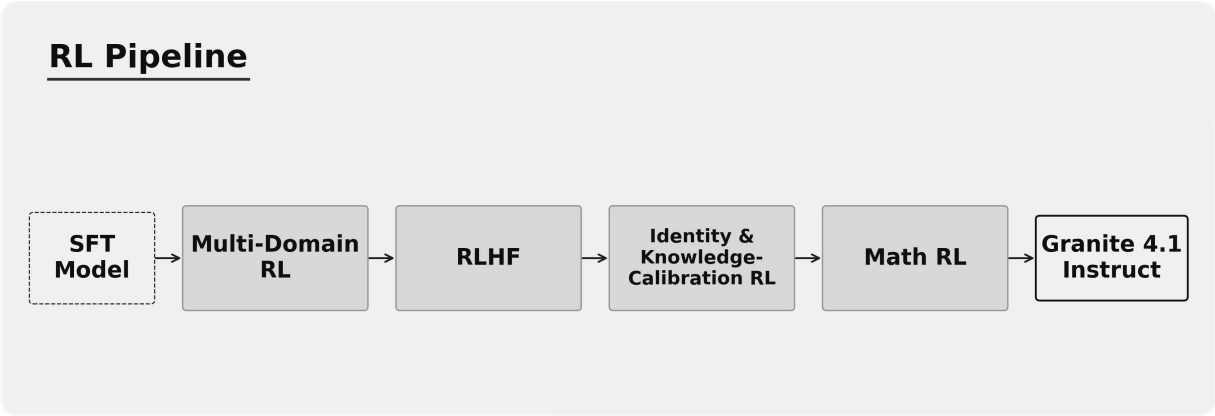


Figure 10: The Granite 4.1 reinforcement learning pipeline consisting of four sequential stages: Multi-domain RL, RLHF, Identity and Knowledge-calibration RL, and Math RL.

[🔗](#) **Multi-domain RL**

In this stage, the model is trained jointly on a unified mixture of data drawn from multiple domains. Every gradient update therefore reflects the full diversity of tasks, which prevents catastrophic forgetting, boosts

overall benchmark performance, and minimizes regressions on any individual task.

The different domains covered in this stage include:

Domain	Description
Math	Mathematical reasoning and computation
Science	Scientific knowledge and reasoning
Logical Reasoning	Deductive and inductive logic
Instruction Following (IF)	Adherence to complex instructions
Structured Output	Structured data output
Text2SQL	Database query generation
Temporal Reasoning	Time-based logic and ordering
General Chat	General conversational quality
In-context Learning	Learning from in-context examples

During this stage, we trained the models on 45,504 unique prompts (averaged across all Granite 4.1 models) and found that a learning rate of $5e-7$ with a KL-loss coefficient (β) of 0.05 performed best for multi-domain reinforcement learning.

 RLHF

To further improve the model's helpfulness and chat ability, we train our model on generic-chat prompts using a multilingual scalar reward model. With this stage, we observed an average improvement of **~18.9 points** (averaged across the three Granite 4.1 models) in Alpaca-Eval compared to the SFT checkpoints.

To mitigate policy drift from its previously learned knowledge, we use a conservative learning rate of $3e-7$ and higher KL-loss coeff β of 0.09 in this stage. We use an average of 17,920 unique prompts in this RLHF stage.

[Identity & Knowledge-Calibration RL](#)

In this stage, we train the model for a few steps (~40 training steps) on identity and knowledge calibration prompts. We observed that this small training stage significantly improves the model's self-identification capabilities.

Similar to the RLHF stage, we used a learning rate of $3e-7$ and KL-loss coeff β of 0.09 , and we use 1728 unique prompts in this stage.

[Math RL](#)

During our RL training, we found that the RLHF stage causes a drop in math benchmark scores (e.g., in GSM8K, DeepMind-Math). The Math RL stage enables the model to recover from this drop and surpasses the original SFT performance on math benchmarks: **~3.8 points** on average for GSM8K, and **~23.48 points** on average for DeepMind-Math. We use an average of 13,504 unique prompts in this stage and similar to the multi-domain RL stage, we used a learning rate of $5e-7$ and KL-loss coeff β of 0.05 .

[Results](#)

[Base Model Benchmarks](#)

Benchmark	Metric	3B	8B	30B
General Tasks				
MMLU	5-shot	66.47	73.60	78.44
MMLU-Pro	5-shot, CoT	37.16	44.58	49.51
BBH	3-shot, CoT	63.84	73.83	80.66
AGI EVAL	3-shot	54.32	61.68	69.20
DROP	5-shot	66.04	72.36	78.57
Math Tasks				
GSM8K	8-shot	72.93	73.54	83.78
Minerva Math	4-shot	38.00	43.42	45.66
Code Tasks				
HumanEval	pass@1 (StarCoder)	76.19	79.24	81.52
HumanEval	pass@1	59.76	68.29	69.50
HumanEval+	pass@1	54.27	62.20	61.60
Eval+ Avg		65.94	62.05	63.90
Multilingual Tasks				
MMMLU	5-shot	56.59	64.73	73.36
INCLUDE	5-shot	51.77	57.60	67.07
MGSM	8-shot	58.48	63.68	74.40

[🔗](#) Instruct Model Benchmarks

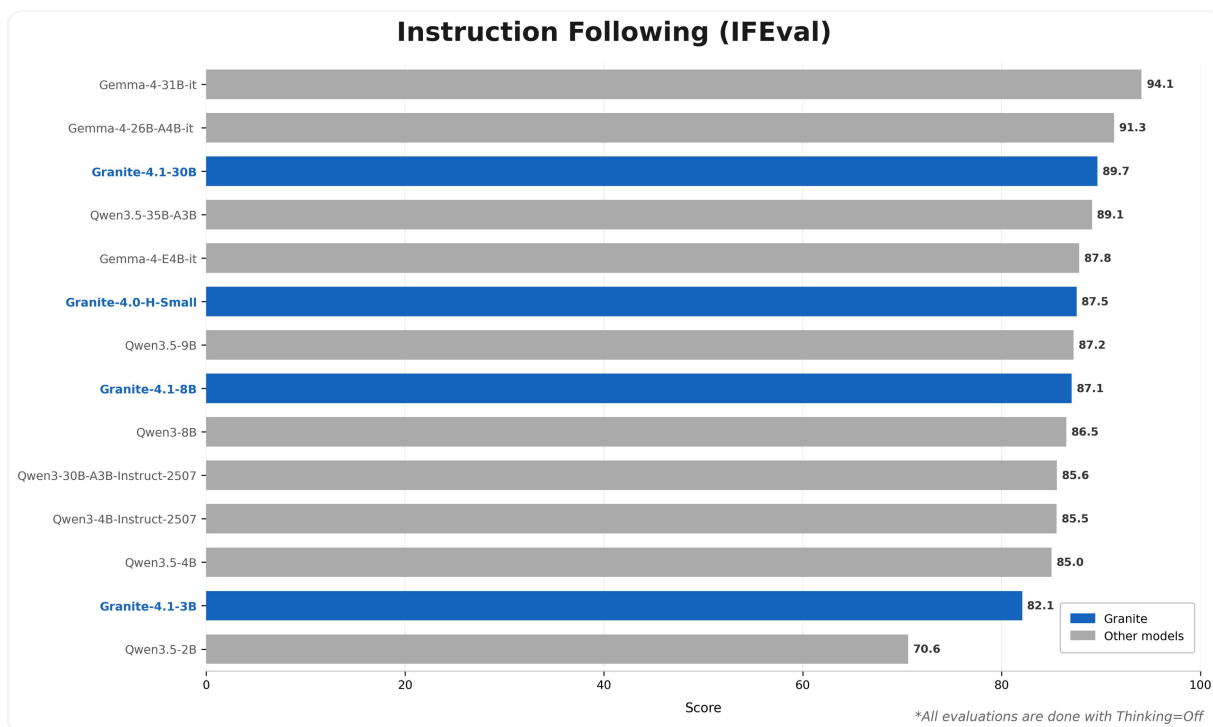
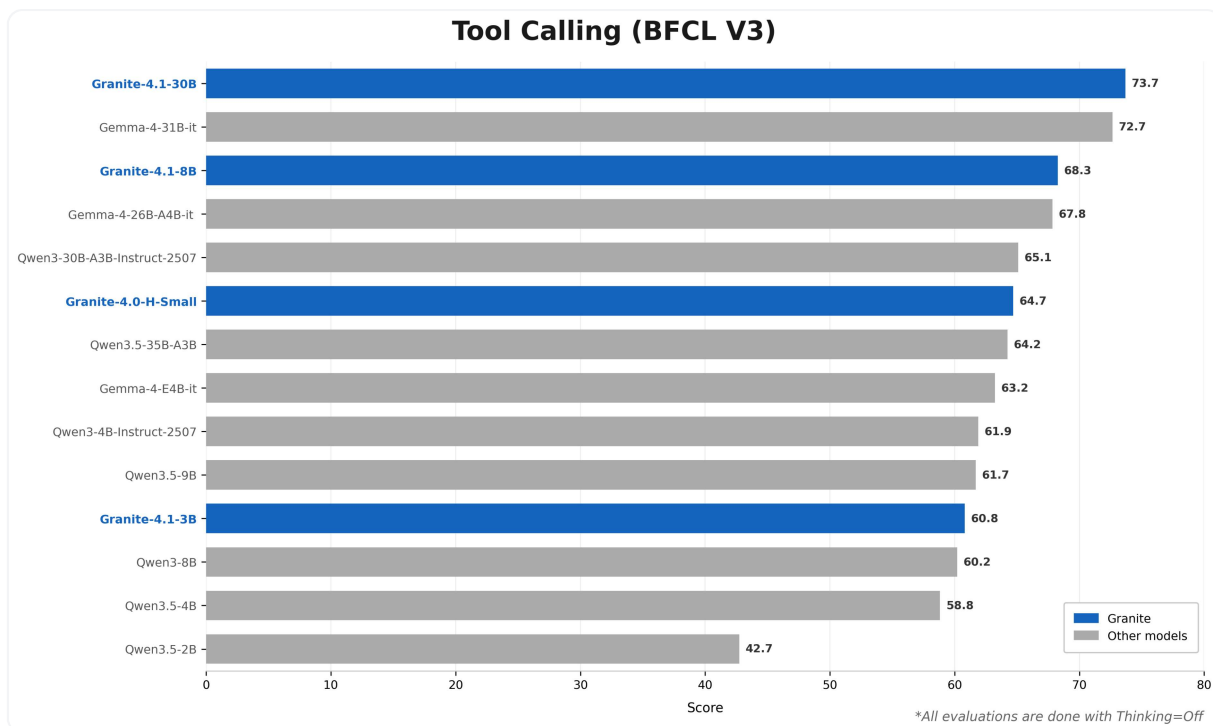
Benchmark	Metric	3B	8B	30B
General Tasks				
MMLU	5-shot	67.02	73.84	80.16
MMLU-Pro	5-shot, CoT	49.83	55.99	64.09
BBH	3-shot, CoT	75.83	80.51	83.74
AGI EVAL	0-shot, CoT	65.16	72.43	77.80
GPQA	0-shot, CoT	31.70	41.96	45.76
SimpleQA		3.68	4.82	6.81
Alignment Tasks				
AlpacaEval 2.0		38.57	50.08	56.16
IFEval Avg		82.30	87.06	89.65
ArenaHard		37.80	68.98	71.02
MTBench Avg		7.53	8.50	8.53
Math Tasks				
GSM8K	8-shot	86.88	92.49	94.16
GSM Symbolic	8-shot	81.32	83.70	75.70
Minerva Math	0-shot, CoT	67.94	80.10	81.32
DeepMind Math	0-shot, CoT	64.64	80.07	81.93
Code Tasks				
HumanEval	pass@1	79.27	87.20	89.63
HumanEval+	pass@1	74.39	80.49	85.98
MBPP	pass@1	61.64	82.54	83.33
MBPP+	pass@1	52.91	70.64	71.69
CRUXEval-O	pass@1	40.75	47.63	55.75

Benchmark	Metric	3B	8B	30B
BigCodeBench	pass@1	32.19	35.00	38.77
MULTIPLE	pass@1	52.54	60.26	62.31
Eval+ Avg	pass@1	67.05	80.21	82.66
Tool Calling				
BFCL v3		60.80	68.27	73.68
Multilingual Tasks				
MMMLU	5-shot	57.61	64.84	73.71
INCLUDE	5-shot	52.05	58.89	67.26
MGSM	8-shot	70.00	82.32	71.12
Safety				
SALAD-Bench		93.95	95.80	96.41
AttaQ		81.88	81.19	85.76
Tulu3 Safety Eval Avg		66.84	75.57	78.19

Supported languages: English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese.

Granite 4.1 Comparison with Leading Open-Source Models

Granite 4.1 delivers competitive instruction-following and tool-calling capabilities without relying on long chains of thought. By avoiding extended reasoning traces, it provides predictable latency, stable token usage, and lower operational cost. This makes Granite 4.1 a production-ready, open-source choice for enterprise workloads where efficiency, reliability, and cost control are critical.



🔗 Granite 4.1-8B vs. Granite 4.0-H-Small (32B-A9B)

A striking result: the Granite 4.1-8B dense model consistently matches or outperforms the previous-generation Granite 4.0-H-Small, a 32B-parameter Mixture-of-Experts model with 9B active parameters.

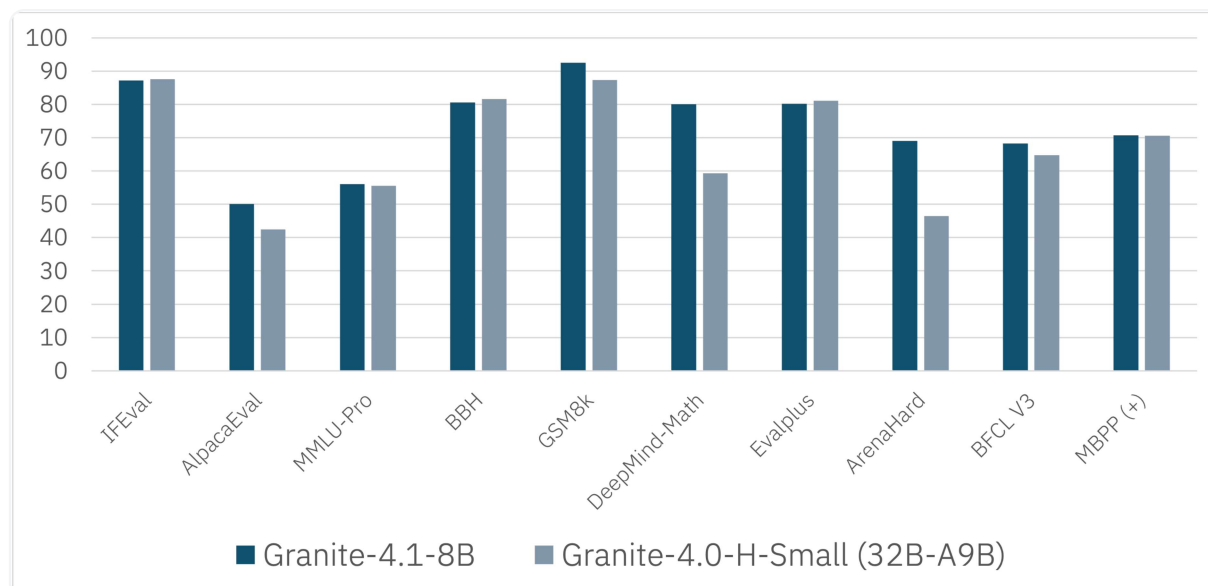


Figure 13: Granite 4.1-8B (dark blue) vs. Granite 4.0-H-Small 32B-A9B (light blue) across benchmarks. The 8B dense model matches or exceeds the larger MoE model on IFEval, AlpacaEval, MMLU-Pro, BBH, GSM8K, DeepMind-Math, Evalplus, ArenaHard, BFCL V3 and MBPP(+).

🔗 Granite 4.1 Model Family Comparison

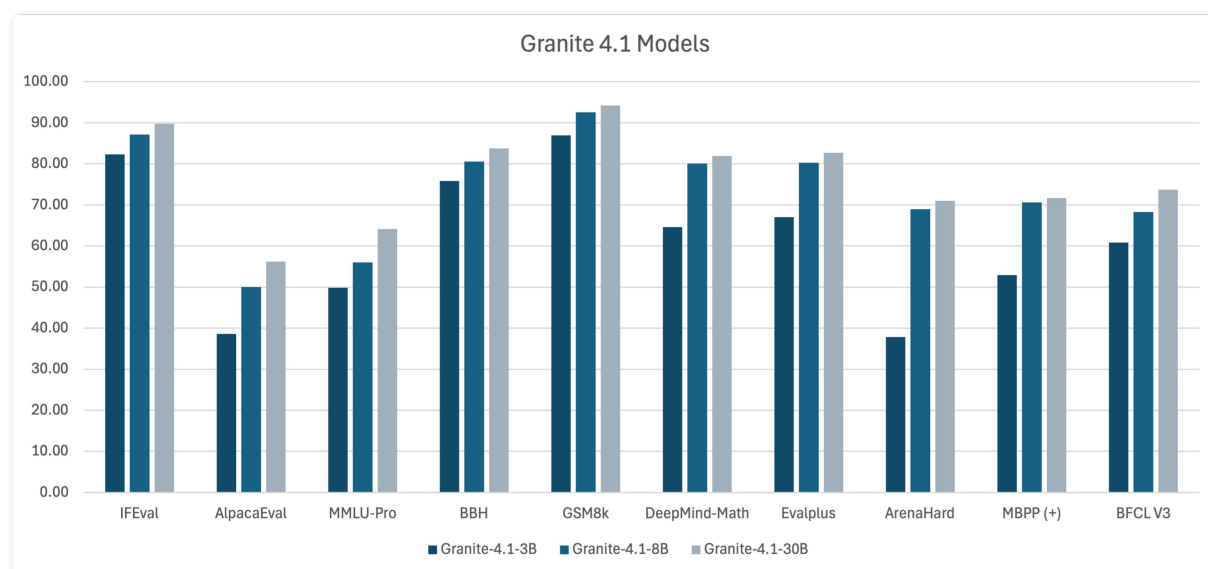


Figure 14: Comparison across the Granite 4.1 family — 30B, 8B, and 3B models. Scores scale predictably with model size, with the 30B model leading across all benchmarks.

🔗 FP8 Quantization

We also released fp8 quantized variants of the Granite 4.1 models, optimized for inference with vLLM. The precision is reduced from 16-bit to 8-bit, resulting in approximately a 50% reduction in both disk footprint and GPU memory usage. Quantization is applied only to the weights and activations of linear operators within the transformer blocks using LLM Compressor, while all other layers are preserved at their original precision.

🔗 Infrastructure

We trained the Granite 4.1 Language Models on an NVIDIA GB200 NVL72 cluster hosted on CoreWeave:

- **Intra-rack communication:** 72-GPU NVLink domain
- **Inter-rack communication:** Non-blocking, full Fat-Tree NDR 400 Gb/s InfiniBand network
- **Scale:** Thousands of GPUs across the cluster

This infrastructure provides the scalable, high-bandwidth interconnect needed for efficient distributed training at the token volumes required (15T+ tokens across pre-training alone).

🔗 Getting Started

Granite 4.1 models are available under the **Apache 2.0 license**. Here's how to get started with the 30B instruct model with tool calling example:

```
pip install torch torchvision torchaudio
pip install accelerate
```

```
pip install transformers
```

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

device = "cuda"
model_path = "ibm-granite/granite-4.1-30b"
tokenizer = AutoTokenizer.from_pretrained(model_path)
# drop device_map if running on CPU
model = AutoModelForCausalLM.from_pretrained(model_path, device_map=device)
model.eval()

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_weather",
            "description": "Get the current weather for a specified city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "Name of the city"
                    }
                }
            },
            "required": ["city"]
        }
    }
]

# change input text as desired
chat = [
    { "role": "user", "content": "What's the weather like in London right now"
}
chat = tokenizer.apply_chat_template(chat, \
                                    tokenize=False, \
                                    tools=tools, \
                                    add_generation_prompt=True)
```

```
# tokenize the text
input_tokens = tokenizer(chat, return_tensors="pt").to(device)
# generate output tokens
output = model.generate(**input_tokens,
                        max_new_tokens=100)
# decode output tokens into text
output = tokenizer.batch_decode(output)
# print output
print(output[0])
```

Expected Output:

```
<|start_of_role|>system<|end_of_role|>You are a helpful assistant with access
to a set of tools that you can use to answer the user's question.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "get_current_weather", "description":
"Get the current weather in a given location."}}
</tools>

For each tool call, return a json object with function name and arguments with
in <tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>. If a tool does not exist in the provided list of tools, notify
the user.

<|start_of_role|>user<|end_of_role|>What's the weather like in London right now?
<|start_of_role|>assistant<|end_of_role|><tool_call>
{"name": "get_current_weather", "arguments": {"city": "London"}}
</tool_call><|end_of_text|>
```

Resources:

- [Granite 4.1 HF Collection](#)
- [PRISM: Demystifying Retention and Interaction in Mid-Training](#)
- [GitHub: ibm-granite/granite-4.1-language-models](#)
- [Granite Documentation](#)
- [Granite Community Resources](#)

Granite 4.1 marks a significant step forward for high-quality, open-source language models. By prioritizing data quality and rigor at every stage—from pre-training curation to supervised fine-tuning and multi-stage reinforcement learning—we deliver a substantially improved post-training pipeline. The result is stronger instruction following, tool use, and conversational performance, showing that carefully trained dense 8B models can rival much larger MoE architectures. We're excited to see how the community adopts and builds on these models.

Papers mentioned in this article 1

PRISM: Demystifying Retention and Interaction in Mid-Training

 Paper • 2603.17074 • Published Mar 17 •  2

Collections mentioned in this article 1

Granite 4.1 Language Models  Collection

Efficient language models for multili... • 6 items • Updated Apr 29 •  55

More from this author

Granite Embedding Multilingual R2: Open Apache 2.0 Multilingual Embeddings with 32K Context — Best Sub-100M Retrieval Quality

 [ibm-granite](#) Enterprise

△ 32 May 15, 2026

Granite 4.0 3B Vision: Compact Multimodal Intelligence for Enterprise Documents

 [ibm-granite](#) Enterprise

△ 34 March 31, 2026

Community

Edit

Preview

Start discussing this article

 Tap or paste here to upload images

 Comment

 System theme

Company

TOS

Privacy

About

Careers

Website

Models

Datasets

Spaces

Pricing

Docs

